Nagar Yuwak Shikshan Sanstha's
Yeshwantrao Chavan College of Engineering
(An Autonomous Institution affiliated to Rashtrasant Tukadoji Maharaj Nagpur University)
Hingna Road, Wanadongri, Nagpur - 441 110
NAAC A++
Ph.: 07104-237919, 234623, 329249, 329250 Fax: 07104-232376, Website: www.ycce.edu
Department of Computer Technology

Vision of the Department

To be a well-known centre for pursuing computer education through innovative pedagogy, value-based education and industry collaboration.

Mission of the Department

To establish learning ambience for ushering in computer engineering professionals in core and multidisciplinary area by developing Problem-solving skills through emerging technologies.

Session 2025-2026

Vision: *To be a well-known centre for pursuing computer education through innovative pedagogy, value-based education and industry collaboration*	**Mission:** *To establish learning ambience for ushering in computer engineering professionals in core and multidisciplinary area by developing Problem-solving skills through emerging technologies.*

Program Educational Objectives of the program (PEO): (broad statements that describe the professional and career accomplishments)

PEO1	**Preparation**	**P: Preparation**	**Pep-CL abbreviation pronounce as Pep-si-lL easy to recall**
PEO2	**Core Competence**	**E: Environment (Learning Environment)**	
PEO3	**Breadth**	**P: Professionalism**	
PEO4	**Professionalism**	**C: Core Competence**	
PEO5	**Learning Environment**	**L: Breadth (Learning in diverse areas)**	

Program Outcomes (PO): (statements that describe what a student should be able to do and know by the end of a program)

Keywords of POs:
Engineering knowledge, Problem analysis, Design/development of solutions, Conduct Investigations of Complex Problems, Engineering Tool Usage, The Engineer and The World, Ethics, Individual and Collaborative Team work, Communication, Project Management and Finance, Life-Long Learning

PSO Keywords: Cutting edge technologies, Research

"I am an engineer, and I know how to apply engineering knowledge to investigate, analyse and design solutions to complex problems using tools for entire world following all ethics in a collaborative way with proper management skills throughout my life." *to contribute to the development of cutting-edge technologies and Research*.

Integrity: I will adhere to the Laboratory Code of Conduct and ethics in its entirety.

SHEIKH WASIMUDDIN

Name and Signature of Student and Date
(Signature and Date in Handwritten)

Sheikh

21/03/26

Nagar Yuwak Shikshan Sanstha's
Yeshwantrao Chavan College of Engineering
(An Autonomous Institution affiliated to Rashtrasant Tukadoji Maharaj Nagpur University)
Hingna Road, Wanadongri, Nagpur - 441 110
NAAC A++
Ph.: 07104-237919, 234623, 329249, 329250 Fax: 07104-232376, Website: www.ycce.edu
Department of Computer Technology

Vision of the Department

To be a well-known centre for pursuing computer education through innovative pedagogy, value-based education and industry collaboration.

Mission of the Department

To establish learning ambience for ushering in computer engineering professionals in core and multidisciplinary area by developing Problem-solving skills through emerging technologies.

Session	**2025-26 (EVEN)**	**Course Name**	**JAVA FSD Lab**
Semester	**6**	**Course Code**	**23IOT1607**
Roll No	**63**	**Name of Student**	**SHEIKH WASIMUDDIN**

Practical Number	**08**
Course Outcome	1. Apply OOP concepts to solve the real-life problem. 2. Use the database concepts to integrate in giving OOP oriented solution. 3. Develop full-stack applications integrating frontend, backend, and database with security. 4. Design and develop mini-project with frontend, backend, and database integration, including authentication and testing.
Aim	Mini Project Activity using Hibernate.
Problem Definition	**Hospital Management System**
Theory (100 words)	MediTrack Hospital Management System is a web-based application developed using Java, JSP, Servlets, Hibernate, Maven, and MySQL. The main objective of this project is to simplify and automate basic hospital operations such as managing patients, doctors, and appointments. The system provides an easy-to-use interface where users can add patient details, register doctors, and schedule appointments efficiently. Hibernate ORM is used to connect the Java application with the MySQL database, reducing the complexity of SQL queries and improving data management. JSP and Servlets handle the front-end and request processing, while Maven is used for dependency management and project build automation. This project demonstrates the practical implementation of CRUD operations in a real-world healthcare domain. It helps in reducing manual paperwork, improving record accuracy, and ensuring faster access to hospital data. Overall, it is a simple yet effective mini-project to understand Java web development with Hibernate.

Procedure and Execution (100 Words)	**1.** Install and set up Java, Maven, Apache Tomcat, MySQL, and IntelliJ IDEA/Eclipse to prepare the development environment for the project. **2.** Create a new Maven-based Dynamic Web Project for the Hospital Management System and define the project structure properly. **3.** Configure the required dependencies in pom.xml, including Hibernate Core, MySQL Connector, Servlet API, and JSP API for web development and database connectivity. **4.** Create the main project structure with packages such as entity, dao, servlet, and util to maintain clean and organized code. **5.** Develop the entity classes such as Patient, Doctor, and Appointment to represent hospital-related data in the database. **6.** Configure Hibernate in hibernate.cfg.xml by setting the database URL, username, password, Hibernate dialect, and mapping all entity classes. **7.** Create the Hibernate utility class (HibernateUtil) to build and manage the SessionFactory for database sessions. **8.** Implement DAO classes for Patient, Doctor, and Appointment to perform CRUD operations like insert, fetch, and manage records efficiently. **9.** Develop servlet classes to handle user requests such as adding patients, adding doctors, booking appointments, and viewing records. **10.** Design JSP pages such as index.jsp, add-patient.jsp, add-doctor.jsp, add-appointment.jsp, and view pages for displaying hospital data. **11.** Apply CSS styling to create a simple, clean, and user-friendly interface for hospital management operations. **12.** Configure web.xml to define the welcome page and ensure proper web application deployment on Apache Tomcat. **13.** Create the MySQL database (hospital_db) and allow

Hibernate to automatically generate the required tables using hbm2ddl.auto=update.

14. Build the project using Maven (mvn clean package) to generate the WAR file for deployment.
15. Deploy the generated WAR file on Apache Tomcat, start the server, and test all modules such as patient management, doctor management, and appointment booking in the browser using http://localhost:8080/MediTrack-Hospital-WebApp.

CODE:

1. AppointmentDAO.java

```
package com.medtrack.dao;

import com.medtrack.entity.Appointment;
import com.medtrack.util.HibernateUtil;
import org.hibernate.Session;
import org.hibernate.Transaction;

import java.util.List;

public class AppointmentDAO {

    public void saveAppointment(Appointment appointment) {
        Transaction tx = null;
        try (Session session =
HibernateUtil.getSessionFactory().openSession()) {
            tx = session.beginTransaction();
            session.save(appointment);
            tx.commit();
        } catch (Exception e) {
            if (tx != null) tx.rollback();
            e.printStackTrace();
        }
    }

    public List<Appointment> getAllAppointments() {
        try (Session session =
HibernateUtil.getSessionFactory().openSession()) {
            return session.createQuery("from Appointment",
```

```
Appointment.class).list();
        }
    }
}
```

2. DoctorDAO.java

```
package com.medtrack.dao;

import com.medtrack.entity.Doctor;
import com.medtrack.util.HibernateUtil;
import org.hibernate.Session;
import org.hibernate.Transaction;

import java.util.List;

public class DoctorDAO {

    public void saveDoctor(Doctor doctor) {
        Transaction tx = null;
        try (Session session =
HibernateUtil.getSessionFactory().openSession()) {
            tx = session.beginTransaction();
            session.save(doctor);
            tx.commit();
        } catch (Exception e) {
            if (tx != null) tx.rollback();
            e.printStackTrace();
        }
    }

    public List<Doctor> getAllDoctors() {
        try (Session session =
HibernateUtil.getSessionFactory().openSession()) {
            return session.createQuery("from Doctor", Doctor.class).list();
        }
    }

    public Doctor getDoctorById(int id) {
        try (Session session =
HibernateUtil.getSessionFactory().openSession()) {
            return session.get(Doctor.class, id);
```

```
        }
    }
}
```

3. PatientDAO.java

```
package com.medtrack.dao;

import com.medtrack.entity.Patient;
import com.medtrack.util.HibernateUtil;
import org.hibernate.Session;
import org.hibernate.Transaction;

import java.util.List;

public class PatientDAO {

    public void savePatient(Patient patient) {
        Transaction tx = null;
        try (Session session =
HibernateUtil.getSessionFactory().openSession()) {
            tx = session.beginTransaction();
            session.save(patient);
            tx.commit();
        } catch (Exception e) {
            if (tx != null) tx.rollback();
            e.printStackTrace();
        }
    }

    public List<Patient> getAllPatients() {
        try (Session session =
HibernateUtil.getSessionFactory().openSession()) {
            return session.createQuery("from Patient", Patient.class).list();
        }
    }

    public Patient getPatientById(int id) {
        try (Session session =
HibernateUtil.getSessionFactory().openSession()) {
            return session.get(Patient.class, id);
        }
    }
```

```
}

4.Appointment.java
package com.medtrack.entity;

import javax.persistence.*;

@Entity
@Table(name = "appointments")
public class Appointment {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private int appointmentId;

    @ManyToOne
    @JoinColumn(name = "patient_id")
    private Patient patient;

    @ManyToOne
    @JoinColumn(name = "doctor_id")
    private Doctor doctor;

    private String appointmentDate;

    public Appointment() {}

    public Appointment(Patient patient, Doctor doctor, String
appointmentDate) {
        this.patient = patient;
        this.doctor = doctor;
        this.appointmentDate = appointmentDate;
    }

    public int getAppointmentId() {
        return appointmentId;
    }

    public void setAppointmentId(int appointmentId) {
        this.appointmentId = appointmentId;
    }

    public Patient getPatient() {
```

```
        return patient;
    }

    public void setPatient(Patient patient) {
        this.patient = patient;
    }

    public Doctor getDoctor() {
        return doctor;
    }

    public void setDoctor(Doctor doctor) {
        this.doctor = doctor;
    }

    public String getAppointmentDate() {
        return appointmentDate;
    }

    public void setAppointmentDate(String appointmentDate) {
        this.appointmentDate = appointmentDate;
    }
}
```

5. Doctor.java

```
package com.medtrack.entity;

import javax.persistence.*;

@Entity
@Table(name = "doctors")
public class Doctor {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private int doctorId;

    private String name;
    private String specialization;

    public Doctor() {}

    public Doctor(String name, String specialization) {
```

```
        this.name = name;
        this.specialization = specialization;
    }

    public int getDoctorId() {
        return doctorId;
    }

    public void setDoctorId(int doctorId) {
        this.doctorId = doctorId;
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public String getSpecialization() {
        return specialization;
    }

    public void setSpecialization(String specialization) {
        this.specialization = specialization;
    }
}
```

6.Patient.java

```
package com.medtrack.entity;

import javax.persistence.*;

@Entity
@Table(name = "patients")
public class Patient {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private int patientId;
```

```
private String name;
private int age;
private String gender;
private String disease;

public Patient() {}

public Patient(String name, int age, String gender, String disease) {
    this.name = name;
    this.age = age;
    this.gender = gender;
    this.disease = disease;
}

public int getPatientId() {
    return patientId;
}

public void setPatientId(int patientId) {
    this.patientId = patientId;
}

public String getName() {
    return name;
}

public void setName(String name) {
    this.name = name;
}

public int getAge() {
    return age;
}

public void setAge(int age) {
    this.age = age;
}

public String getGender() {
    return gender;
}

public void setGender(String gender) {
```

```
        this.gender = gender;
    }

    public String getDisease() {
        return disease;
    }

    public void setDisease(String disease) {
        this.disease = disease;
    }
}
```

7. AddAppointmentServlet.java

```
package com.medtrack.servlet;

import com.medtrack.dao.AppointmentDAO;
import com.medtrack.dao.DoctorDAO;
import com.medtrack.dao.PatientDAO;
import com.medtrack.entity.Appointment;
import com.medtrack.entity.Doctor;
import com.medtrack.entity.Patient;

import javax.servlet.ServletException;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.*;
import java.io.IOException;

@WebServlet("/addAppointment")
public class AddAppointmentServlet extends HttpServlet {
    protected void doPost(HttpServletRequest request,
HttpServletResponse response)
            throws ServletException, IOException {

        int patientId = Integer.parseInt(request.getParameter("patientId"));
        int doctorId = Integer.parseInt(request.getParameter("doctorId"));
        String appointmentDate =
request.getParameter("appointmentDate");

        Patient patient = new PatientDAO().getPatientById(patientId);
        Doctor doctor = new DoctorDAO().getDoctorById(doctorId);

        if (patient != null && doctor != null) {
```

```
            Appointment appointment = new Appointment(patient, doctor,
appointmentDate);
            new AppointmentDAO().saveAppointment(appointment);
        }

        response.sendRedirect("view-appointments");
    }
}
```

8. AddDoctorServlet.java

```
package com.medtrack.servlet;

import com.medtrack.dao.DoctorDAO;
import com.medtrack.entity.Doctor;

import javax.servlet.ServletException;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.*;
import java.io.IOException;

@WebServlet("/addDoctor")
public class AddDoctorServlet extends HttpServlet {
    protected void doPost(HttpServletRequest request,
HttpServletResponse response)
            throws ServletException, IOException {

        String name = request.getParameter("name");
        String specialization = request.getParameter("specialization");

        Doctor doctor = new Doctor(name, specialization);
        new DoctorDAO().saveDoctor(doctor);

        response.sendRedirect("view-doctors");
    }
}
```

9. AddPatientServlet.java

```
package com.medtrack.servlet;

import com.medtrack.dao.PatientDAO;
import com.medtrack.entity.Patient;

import javax.servlet.ServletException;
```

```
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.*;
import java.io.IOException;

@WebServlet("/addPatient")
public class AddPatientServlet extends HttpServlet {
   protected void doPost(HttpServletRequest request,
HttpServletResponse response)
         throws ServletException, IOException {

      String name = request.getParameter("name");
      int age = Integer.parseInt(request.getParameter("age"));
      String gender = request.getParameter("gender");
      String disease = request.getParameter("disease");

      Patient patient = new Patient(name, age, gender, disease);
      new PatientDAO().savePatient(patient);

      response.sendRedirect("view-patients");
   }
}
```

10. ViewAppointmentsServlet.java

```
package com.medtrack.servlet;

import com.medtrack.dao.AppointmentDAO;
import com.medtrack.entity.Appointment;

import javax.servlet.ServletException;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.*;
import java.io.IOException;
import java.util.List;

@WebServlet("/view-appointments")
public class ViewAppointmentsServlet extends HttpServlet {
   protected void doGet(HttpServletRequest request,
HttpServletResponse response)
         throws ServletException, IOException {

      List<Appointment> appointments = new
AppointmentDAO().getAllAppointments();
      request.setAttribute("appointments", appointments);
```

```
        request.getRequestDispatcher("view-
appointments.jsp").forward(request, response);
    }
}
```

11. ViewDoctorsServlet.java

```
package com.medtrack.servlet;

import com.medtrack.dao.DoctorDAO;
import com.medtrack.entity.Doctor;

import javax.servlet.ServletException;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.*;
import java.io.IOException;
import java.util.List;

@WebServlet("/view-doctors")
public class ViewDoctorsServlet extends HttpServlet {
    protected void doGet(HttpServletRequest request,
HttpServletResponse response)
            throws ServletException, IOException {

        List<Doctor> doctors = new DoctorDAO().getAllDoctors();
        request.setAttribute("doctors", doctors);
        request.getRequestDispatcher("view-doctors.jsp").forward(request,
response);
    }
}
```

12. ViewPatientsServlet.java

```
package com.medtrack.servlet;

import com.medtrack.dao.PatientDAO;
import com.medtrack.entity.Patient;

import javax.servlet.ServletException;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.*;
import java.io.IOException;
import java.util.List;
```

```
@WebServlet("/view-patients")
public class ViewPatientsServlet extends HttpServlet {
   protected void doGet(HttpServletRequest request,
HttpServletResponse response)
         throws ServletException, IOException {

      List<Patient> patients = new PatientDAO().getAllPatients();
      request.setAttribute("patients", patients);
      request.getRequestDispatcher("view-patients.jsp").forward(request,
response);
   }
}
```

13. HibernateUtil.java

```
package com.medtrack.util;

import org.hibernate.SessionFactory;
import org.hibernate.cfg.Configuration;

public class HibernateUtil {

   private static final SessionFactory sessionFactory =
buildSessionFactory();

   private static SessionFactory buildSessionFactory() {
      try {
         return new
Configuration().configure("hibernate.cfg.xml").buildSessionFactory();
      } catch (Throwable ex) {
         System.err.println("SessionFactory creation failed: " + ex);
         throw new ExceptionInInitializerError(ex);
      }
   }

   public static SessionFactory getSessionFactory() {
      return sessionFactory;
   }
}
```

14. hibernate.cfg.xml

```
<?xml version="1.0" encoding="UTF-8"?>
```

```
<!DOCTYPE hibernate-configuration PUBLIC
        "-//Hibernate/Hibernate Configuration DTD 3.0//EN"
        "http://hibernate.sourceforge.net/hibernate-configuration-3.0.dtd">

<hibernate-configuration>
    <session-factory>

        <property
name="hibernate.connection.driver_class">com.mysql.cj.jdbc.Driver</p
roperty>
        <property
name="hibernate.connection.url">jdbc:mysql://localhost:3306/hospital_
db</property>
        <property
name="hibernate.connection.username">root</property>
        <property
name="hibernate.connection.password">7421</property>

        <property
name="hibernate.dialect">org.hibernate.dialect.MySQL8Dialect</prope
rty>
        <property name="hibernate.show_sql">true</property>
        <property name="hibernate.format_sql">true</property>
        <property name="hibernate.hbm2ddl.auto">update</property>

        <mapping class="com.medtrack.entity.Patient"/>
        <mapping class="com.medtrack.entity.Doctor"/>
        <mapping class="com.medtrack.entity.Appointment"/>

    </session-factory>
</hibernate-configuration>
```

15.Style.css

```
body {
    font-family: Arial, sans-serif;
    background: linear-gradient(135deg, #0f172a, #1e3a8a);
    margin: 0;
    padding: 0;
    color: white;
}

.container {
    width: 80%;
```

```
    margin: 50px auto;
    text-align: center;
}

h1, h2 {
    margin-bottom: 30px;
}

.menu {
    display: grid;
    grid-template-columns: repeat(2, 1fr);
    gap: 20px;
    max-width: 700px;
    margin: auto;
}

.menu a, button {
    display: inline-block;
    padding: 15px 20px;
    background: #2563eb;
    color: white;
    text-decoration: none;
    border-radius: 10px;
    border: none;
    cursor: pointer;
    transition: 0.3s;
    font-weight: bold;
}

.menu a:hover, button:hover {
    background: #1d4ed8;
}

.form-container, .table-container {
    width: 80%;
    max-width: 800px;
    margin: 50px auto;
    background: rgba(255,255,255,0.1);
    padding: 30px;
    border-radius: 15px;
    text-align: center;
}
```

```
input {
   width: 90%;
   padding: 12px;
   margin: 10px 0;
   border-radius: 8px;
   border: none;
   font-size: 16px;
}

a {
   color: #93c5fd;
   display: inline-block;
   margin-top: 15px;
   text-decoration: none;
   font-weight: bold;
}

table {
   width: 100%;
   border-collapse: collapse;
   background: white;
   color: black;
   border-radius: 10px;
   overflow: hidden;
}

table th, table td {
   padding: 12px;
   border: 1px solid #ddd;
}

table th {
   background: #2563eb;
   color: white;
}
```

16. Add-appointment.jsp

```
<%@ page contentType="text/html;charset=UTF-8"
language="java" %>
<!DOCTYPE html>
<html>
<head>
```

```
    <title>Book Appointment</title>
    <link rel="stylesheet" href="css/style.css">
</head>
<body>
<div class="form-container">
    <h2>Book Appointment</h2>
    <form action="addAppointment" method="post">
        <input type="number" name="patientId" placeholder="Patient
ID" required>
        <input type="number" name="doctorId" placeholder="Doctor
ID" required>
        <input type="date" name="appointmentDate" required>
        <button type="submit">Book Appointment</button>
    </form>
    <a href="index.jsp">Back to Home</a>
</div>
</body>
</html>
```

17. Add-doctor.jsp

```
<%@ page contentType="text/html;charset=UTF-8"
language="java" %>
<!DOCTYPE html>
<html>
<head>
    <title>Add Doctor</title>
    <link rel="stylesheet" href="css/style.css">
</head>
<body>
<div class="form-container">
    <h2>Add Doctor</h2>
    <form action="addDoctor" method="post">
        <input type="text" name="name" placeholder="Doctor Name"
required>
        <input type="text" name="specialization"
placeholder="Specialization" required>
        <button type="submit">Save Doctor</button>
    </form>
    <a href="index.jsp">Back to Home</a>
</div>
</body>
</html>
```

18. Add-patient.jsp

```
<%@ page contentType="text/html;charset=UTF-8"
language="java" %>
<!DOCTYPE html>
<html>
<head>
   <title>Add Patient</title>
   <link rel="stylesheet" href="css/style.css">
</head>
<body>
<div class="form-container">
   <h2>Add Patient</h2>
   <form action="addPatient" method="post">
      <input type="text" name="name" placeholder="Patient Name"
required>
      <input type="number" name="age" placeholder="Age"
required>
      <input type="text" name="gender" placeholder="Gender"
required>
      <input type="text" name="disease" placeholder="Disease"
required>
      <button type="submit">Save Patient</button>
   </form>
   <a href="index.jsp">Back to Home</a>
</div>
</body>
</html>
```

19. Index.jsp

```
<%@ page contentType="text/html;charset=UTF-8"
language="java" %>
<!DOCTYPE html>
<html>
<head>
   <title>MediTrack Hospital Management</title>
   <link rel="stylesheet" href="css/style.css">
</head>
<body>
<div class="container">
   <h1>MediTrack Hospital Management System</h1>
   <div class="menu">
      <a href="add-patient.jsp">Add Patient</a>
      <a href="view-patients">View Patients</a>
      <a href="add-doctor.jsp">Add Doctor</a>
```

```
        <a href="view-doctors">View Doctors</a>
        <a href="add-appointment.jsp">Book Appointment</a>
        <a href="view-appointments">View Appointments</a>
    </div>
</div>
</body>
</html>
```

20. View-appointments.jsp

```
<%@ page import="java.util.List" %>
<%@ page import="com.medtrack.entity.Appointment" %>
<%
    List<Appointment> appointments = (List<Appointment>)
request.getAttribute("appointments");
%>
<!DOCTYPE html>
<html>
<head>
    <title>View Appointments</title>
    <link rel="stylesheet" href="css/style.css">
</head>
<body>
<div class="table-container">
    <h2>Appointments List</h2>
    <table>
        <tr>

<th>ID</th><th>Patient</th><th>Doctor</th><th>Date</th>
        </tr>
        <%
            if (appointments != null) {
                for (Appointment a : appointments) {
        %>
        <tr>
            <td><%= a.getAppointmentId() %></td>
            <td><%= a.getPatient().getName() %></td>
            <td><%= a.getDoctor().getName() %></td>
            <td><%= a.getAppointmentDate() %></td>
        </tr>
        <%
                }
            }
        %>
    </table>
```

```
    <a href="index.jsp">Back to Home</a>
</div>
</body>
</html>
```

21. View-doctors.jsp

```
<%@ page import="java.util.List" %>
<%@ page import="com.medtrack.entity.Doctor" %>
<%
    List<Doctor> doctors = (List<Doctor>)
request.getAttribute("doctors");
%>
<!DOCTYPE html>
<html>
<head>
    <title>View Doctors</title>
    <link rel="stylesheet" href="css/style.css">
</head>
<body>
<div class="table-container">
    <h2>Doctors List</h2>
    <table>
        <tr>
            <th>ID</th><th>Name</th><th>Specialization</th>
        </tr>
        <%
            if (doctors != null) {
                for (Doctor d : doctors) {
        %>
        <tr>
            <td><%= d.getDoctorId() %></td>
            <td><%= d.getName() %></td>
            <td><%= d.getSpecialization() %></td>
        </tr>
        <%
                }
            }
        %>
    </table>
    <a href="index.jsp">Back to Home</a>
</div>
</body>
</html>
```

22. View-patients.jsp

```
<%@ page import="java.util.List" %>
<%@ page import="com.medtrack.entity.Patient" %>
<%
   List<Patient> patients = (List<Patient>)
request.getAttribute("patients");
%>
<!DOCTYPE html>
<html>
<head>
   <title>View Patients</title>
   <link rel="stylesheet" href="css/style.css">
</head>
<body>
<div class="table-container">
   <h2>Patients List</h2>
   <table>
      <tr>

<th>ID</th><th>Name</th><th>Age</th><th>Gender</th><th>Di
sease</th>
      </tr>
      <%
         if (patients != null) {
            for (Patient p : patients) {
      %>
      <tr>
         <td><%= p.getPatientId() %></td>
         <td><%= p.getName() %></td>
         <td><%= p.getAge() %></td>
         <td><%= p.getGender() %></td>
         <td><%= p.getDisease() %></td>
      </tr>
      <%
            }
         }
      %>
   </table>
   <a href="index.jsp">Back to Home</a>
</div>
</body>
</html>
```

23. .gitignore

```
target/
!.mvn/wrapper/maven-wrapper.jar
!**/src/main/**/target/
!**/src/test/**/target/
.kotlin

### IntelliJ IDEA ###
.idea/modules.xml
.idea/jarRepositories.xml
.idea/compiler.xml
.idea/libraries/
*.iws
*.iml
*.ipr

### Eclipse ###
.apt_generated
.classpath
.factorypath
.project
.settings
.springBeans
.sts4-cache

### NetBeans ###
/nbproject/private/
/nbbuild/
/dist/
/nbdist/
/.nb-gradle/
build/
!**/src/main/**/build/
!**/src/test/**/build/

### VS Code ###
.vscode/

### Mac OS ###
.DS_Store
```

Vision of the Department

To be a well-known centre for pursuing computer education through innovative pedagogy, value-based education and industry collaboration.

Mission of the Department

To establish learning ambience for ushering in computer engineering professionals in core and multidisciplinary area by developing Problem-solving skills through emerging technologies.

24. .project

```
<?xml version="1.0" encoding="UTF-8"?>
<projectDescription>
   <name>MediTrack-Hospital-WebApp</name>
   <comment></comment>
   <projects>
   </projects>
   <buildSpec>
      <buildCommand>
         <name>org.eclipse.jdt.core.javabuilder</name>
         <arguments>
         </arguments>
      </buildCommand>
      <buildCommand>

<name>org.eclipse.wst.common.project.facet.core.builder</name>
         <arguments>
         </arguments>
      </buildCommand>
      <buildCommand>
         <name>org.eclipse.wst.validation.validationbuilder</name>
         <arguments>
         </arguments>
      </buildCommand>
      <buildCommand>
         <name>org.eclipse.m2e.core.maven2Builder</name>
         <arguments>
         </arguments>
      </buildCommand>
   </buildSpec>
   <natures>
      <nature>org.eclipse.jem.workbench.JavaEMFNature</nature>

<nature>org.eclipse.wst.common.modulecore.ModuleCoreNature</
nature>
      <nature>org.eclipse.jdt.core.javanature</nature>
      <nature>org.eclipse.m2e.core.maven2Nature</nature>

<nature>org.eclipse.wst.common.project.facet.core.nature</nature>
      <nature>org.eclipse.wst.jsdt.core.jsNature</nature>
   </natures>
</projectDescription>
```

25. Pom.xml

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
        xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
        xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
        http://maven.apache.org/xsd/maven-4.0.0.xsd">

  <modelVersion>4.0.0</modelVersion>
  <groupId>com.medtrack</groupId>
  <artifactId>MediTrack-Hospital-WebApp</artifactId>
  <version>1.0-SNAPSHOT</version>
  <packaging>war</packaging>

  <dependencies>
    <!-- Servlet API -->
    <dependency>
      <groupId>javax.servlet</groupId>
      <artifactId>javax.servlet-api</artifactId>
      <version>4.0.1</version>
      <scope>provided</scope>
    </dependency>

    <!-- JSP API -->
    <dependency>
      <groupId>javax.servlet.jsp</groupId>
      <artifactId>javax.servlet.jsp-api</artifactId>
      <version>2.3.3</version>
      <scope>provided</scope>
    </dependency>

    <!-- JSTL -->
    <dependency>
      <groupId>javax.servlet</groupId>
      <artifactId>jstl</artifactId>
      <version>1.2</version>
    </dependency>

    <!-- Hibernate -->
    <dependency>
      <groupId>org.hibernate</groupId>
      <artifactId>hibernate-core</artifactId>
      <version>5.6.15.Final</version>
    </dependency>
```

Vision of the Department

To be a well-known centre for pursuing computer education through innovative pedagogy, value-based education and industry collaboration.

Mission of the Department

To establish learning ambience for ushering in computer engineering professionals in core and multidisciplinary area by developing Problem-solving skills through emerging technologies.

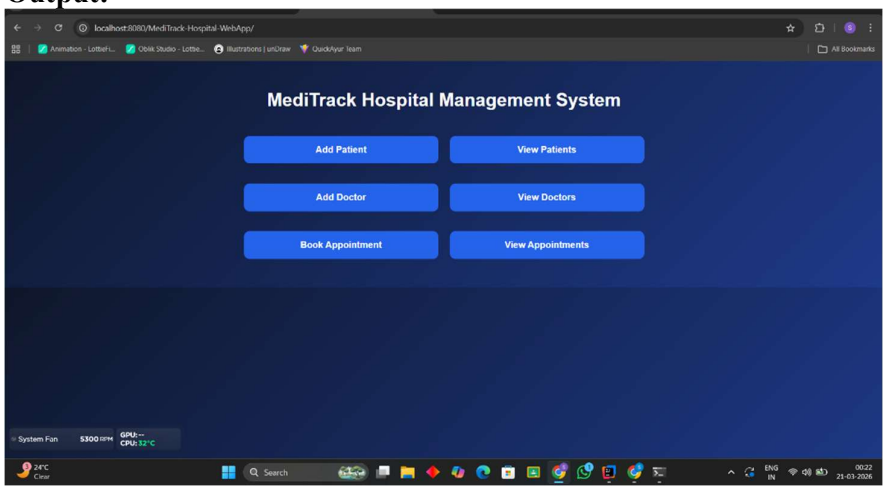
```
        <!-- MySQL Connector -->
        <dependency>
            <groupId>mysql</groupId>
            <artifactId>mysql-connector-java</artifactId>
            <version>8.0.33</version>
        </dependency>

        <!-- JPA API -->
        <dependency>
            <groupId>javax.persistence</groupId>
            <artifactId>javax.persistence-api</artifactId>
            <version>2.2</version>
        </dependency>

        <!-- Logging -->
        <dependency>
            <groupId>org.slf4j</groupId>
            <artifactId>slf4j-simple</artifactId>
            <version>1.7.36</version>
        </dependency>
    </dependencies>

    <build>
        <finalName>MediTrack-Hospital-WebApp</finalName>
    </build>
</project>
```

Output:

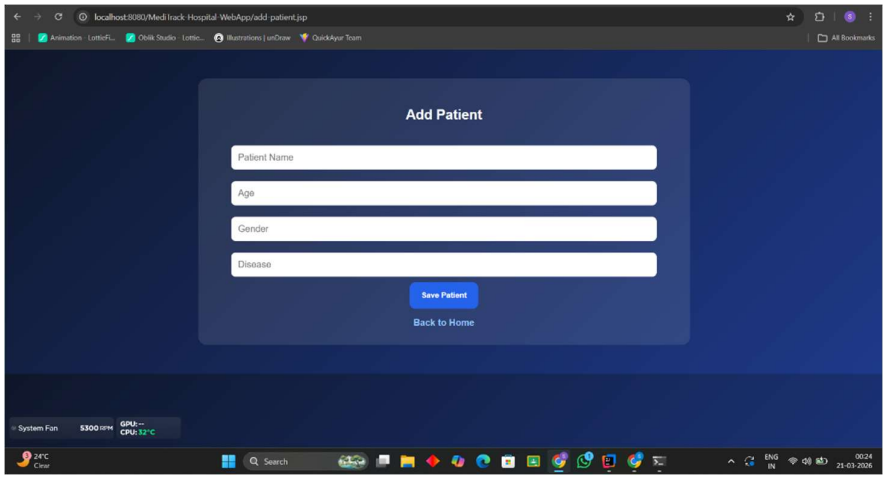


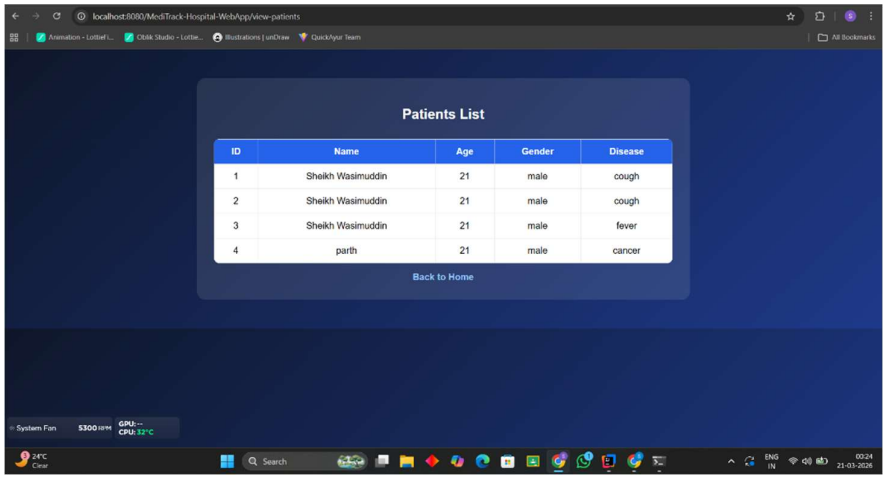
Vision of the Department

To be a well-known centre for pursuing computer education through innovative pedagogy, value-based education and industry collaboration.

Mission of the Department

To establish learning ambience for ushering in computer engineering professionals in core and multidisciplinary area by developing Problem-solving skills through emerging technologies.



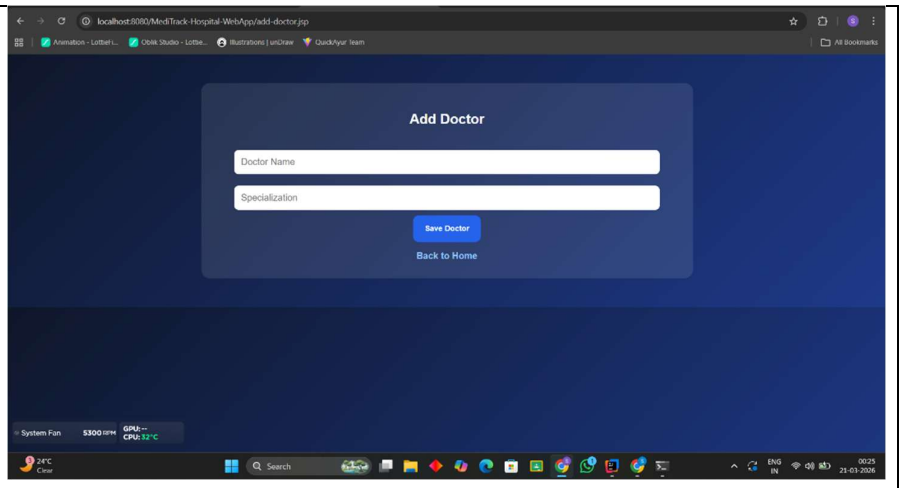


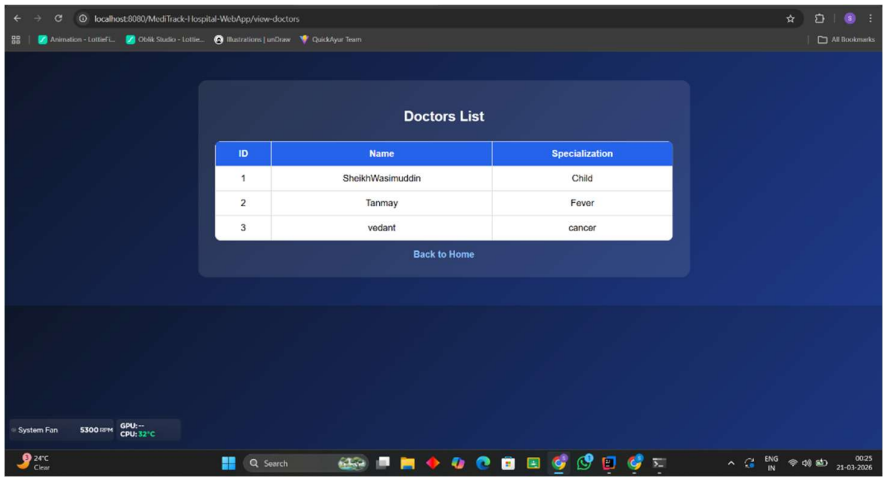
Vision of the Department

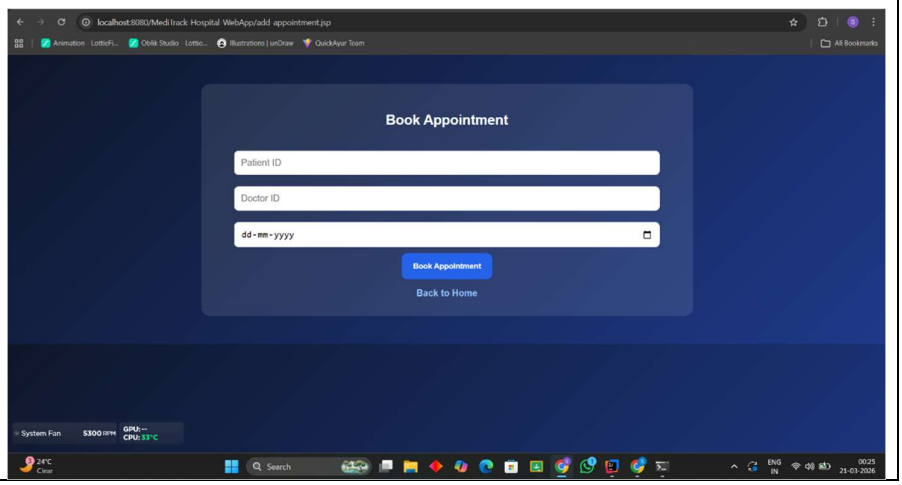
To be a well-known centre for pursuing computer education through innovative pedagogy, value-based education and industry collaboration.

Mission of the Department

To establish learning ambience for ushering in computer engineering professionals in core and multidisciplinary area by developing Problem-solving skills through emerging technologies.





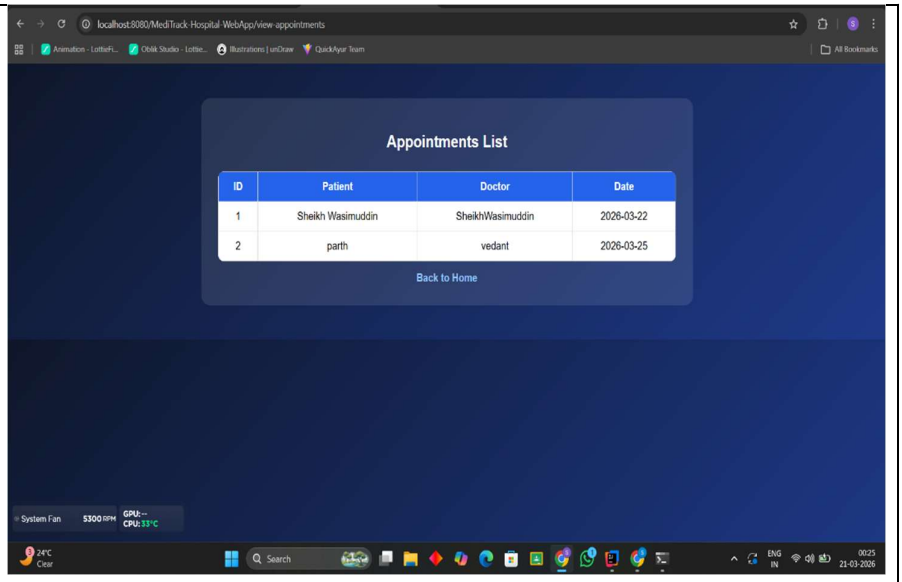


Vision of the Department

To be a well-known centre for pursuing computer education through innovative pedagogy, value-based education and industry collaboration.

Mission of the Department

To establish learning ambience for ushering in computer engineering professionals in core and multidisciplinary area by developing Problem-solving skills through emerging technologies.

	
Output Analysis	The output of the Hospital Management System demonstrates that the application successfully performs the core functionalities of managing hospital records in a web-based environment. When the project is executed on Apache Tomcat, the home page is displayed with options such as Add Patient, View Patients, Add Doctor, View Doctors, Book Appointment, and View Appointments. This confirms that the web interface is loaded correctly and the application is deployed successfully. When patient details are entered through the Add Patient form, the system stores the data in the MySQL database using Hibernate ORM and displays a success response. Similarly, doctor information can be added through the Add Doctor module and saved successfully. The View Patients and View Doctors pages fetch records dynamically from the database and display them in tabular format, proving that database retrieval is functioning correctly. The Book Appointment module allows linking a patient with a doctor by entering their IDs and appointment date. The appointment details are stored in the database and are later displayed in the View Appointments page. This confirms that

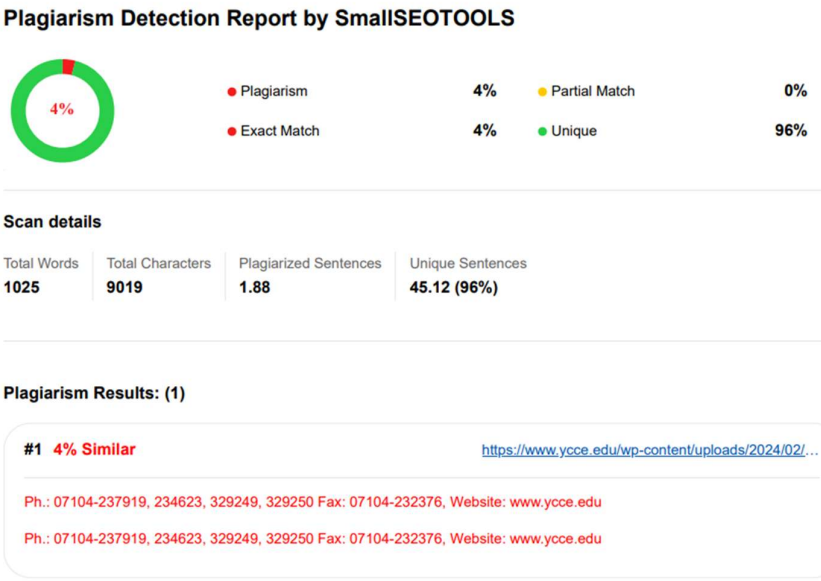
	the relationships between entities are managed properly. Overall, the project output verifies that the application performs CRUD-related operations effectively, provides proper user interaction, and reduces manual hospital record management.
Link of student Github profile where lab assignment has been uploaded	https://github.com/sheikhwasimuddin/Hospital-Management-System-Hibernate
Conclusion	The Hospital Management System project was successfully developed as a web-based application using Java, JSP, Servlets, Hibernate, Maven, MySQL, and Apache Tomcat. The project fulfills its main objective of simplifying and automating basic hospital operations such as managing patients, doctors, and appointments. Through this system, users can efficiently add patient records, register doctors, and book appointments without relying on manual paperwork. The integration of Hibernate ORM with MySQL provides efficient database handling and reduces the complexity of direct SQL queries. JSP and Servlets ensure smooth interaction between the user interface and the back-end logic, while Maven helps in managing dependencies and building the project. The successful execution of all modules proves that the system works correctly and can be used as a basic model for hospital data management. Overall, this project demonstrates the practical implementation of Java web development concepts and provides a strong foundation for developing larger healthcare management systems in the future.

Vision of the Department

To be a well-known centre for pursuing computer education through innovative pedagogy, value-based education and industry collaboration.

Mission of the Department

To establish learning ambience for ushering in computer engineering professionals in core and multidisciplinary area by developing Problem-solving skills through emerging technologies.

Plag Report (Similarity index < 12%)	 **Plagiarism Detection Report by SmallSEOTOOLS** 4% Plagiarism 4% Partial Match 0% Exact Match 4% Unique 96% **Scan details** Total Words 1025 | Total Characters 9019 | Plagiarized Sentences 1.88 | Unique Sentences 45.12 (96%) **Plagiarism Results: (1)** #1 4% Similar https://www.ycce.edu/wp-content/uploads/2024/02/... Ph.: 07104-237919, 234623, 329249, 329250 Fax: 07104-232376, Website: www.ycce.edu Ph.: 07104-237919, 234623, 329249, 329250 Fax: 07104-232376, Website: www.ycce.edu
Date	**21/03/2026**